



C Programming PROJECT REPORT

Title Page:

- Title: Simple Console-based Chatbot in C
- Course: Programming in C (CSEG1041)
- Submitted by: Vipranish Sharma, SAP: 590022121
- Submitted to: Dr. Tanu Singh
- Date: 30th November, 2025

Abstract:

This project implements a simple rule-based chatbot in C that interacts with the user through the console. The chatbot greets the user, asks for their name, and responds to a predefined set of commands such as greetings, time query, facts, jokes, and motivational messages. Randomization is used to vary responses for a more natural feel. The system demonstrates basic concepts of C programming, including input/output handling, string processing, conditional statements, loops, and use of standard libraries.

Problem Definition:

The objective of this project is to design and implement a basic text-based chatbot that can simulate a simple conversation with a user on the command line. The chatbot should:

- Accept the user's name and personalise some responses.
- Recognize a fixed set of commands (e.g., "hi", "how are you", "time", "joke", "bye") and produce appropriate replies.
- Provide random variations for some responses to avoid repetitive output.
- Run continuously in a loop until the user chooses to exit with the "bye" command.

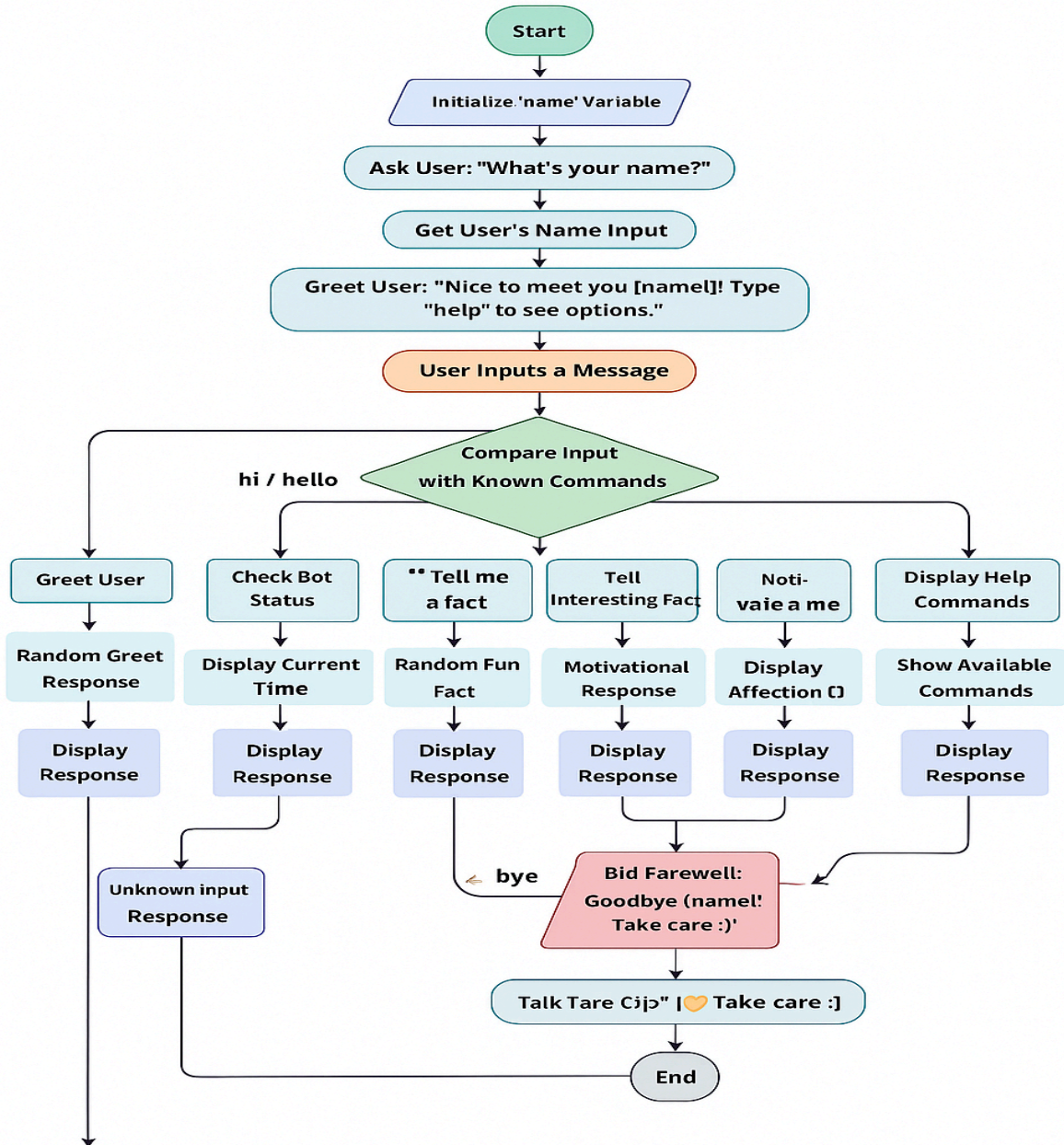
System Design (Flowcharts, Algorithms):

1) Algorithm:

- Start
- Initialize random number generator using current time.
- Clear the screen and greet the user.
- Ask for and read the user's name, then remove the trailing newline.
- Show command hint ("Type help for commands").
- Enter an infinite loop:
 - Prompt "You: " and read a line of user input into msg.
 - Remove the trailing newline from msg.
 - Clear the screen.
 - Compare msg with known commands using string comparison:
 - If greeting ("hi" / "hello"): generate a random number and print one of several greeting replies.
 - If "how are you": generate a random reply about bot status.
 - If "time": get current system time and display it.
 - If "tell me a fact": choose one of several predefined facts at random.
 - If "motivate me": choose one of several motivational lines at random.
 - If "love you bot": print a personalized friendly reply using the user's name.
 - If "joke": choose one of several humorous replies at random.
 - If "help": print the list of supported commands.
 - If "bye": print a goodbye message with the user's name and break the loop.
 - Else: print a generic "don't understand / error" style message, chosen randomly.
- End program.

2) Flowchart:

< C Chatbot Flowchart



Implementation Details (with snippets):

1. Including Required Libraries

```
#include <stdio.h>
#include <string.h>
```

```
#include <stdlib.h>
#include <time.h>
```

What this does:

- `stdio.h` → used for input/output (`printf`, `fgets`)
 - `string.h` → used for string functions (`strcmp`, `strcspn`)
 - `stdlib.h` → used for `rand()`, `srand()`
 - `time.h` → used to get system time and seed random numbers
-

2. Global Variable for Username

```
char name[50];
```

What this does:

Stores the user's name so the chatbot can use it later (example: "Goodbye John").

3. Random Reply Function

```
void randomReply(const char *replies[], int size) {
    printf("C-Bot: %s\n", replies[rand() % size]);
}
```

What this does:

- Selects a **random reply** from an array of strings.
- Prints that reply with the prefix "C-Bot:".

This function makes the chatbot feel more natural instead of giving the same answer every time.

4. Main Function Start

```
int main() {  
    srand(time(NULL));  
    char input[200];
```

What this does:

- `srand(time(NULL));` → seeds random number generator so replies are always different.
- Declares `input[ ]` to store user messages.

5. Asking for User's Name

```
printf("C-Bot: What's your name?\n");  
printf("You: ");  
fgets(name, sizeof(name), stdin);  
name[strcspn(name, "\n")] = 0;
```

What this does:

- Asks user to enter their name.
- Reads input using `fgets`.
- Removes the newline character `\n` that `fgets` stores.

6. Greeting the User

```
printf("C-Bot: Nice to meet you, %s! Type 'help' to see options.\n\n",  
name);
```

What this does:

Prints a personalized greeting and tells the user about the help command.

7. Chat Loop

```
while (1) {  
    printf("You: ");  
    fgets(input, sizeof(input), stdin);  
    input[strcspn(input, "\n")] = 0;
```

What this does:

- Runs forever until user types "bye".
 - Reads user's message each time.
-

8. Handling “hi / hello”

```
if (strcmp(input, "hi") == 0 || strcmp(input, "hello") == 0) {  
    const char *greet[] = {"Hey there! :) ", "Hello! How's it going?",  
    "Hi! Happy to chat :)"};  
    randomReply(greet, 3);  
}
```

What this does:

If the user says "hi" or "hello", chatbot gives a random greeting.

9. Handling “how are you”

```
else if (strcmp(input, "how are you") == 0) {  
    const char *ans[] = {  
        "I'm good! Just chilling in the RAM :D",
```

```
        "Fantastic! Thanks for asking.",
        "Running at 0%% bugs today XD"
    };
    randomReply(ans, 3);
}
```

What this does:

Gives a fun and random emotional response.

10. Displaying System Time

```
else if (strcmp(input, "time") == 0) {
    time_t t;
    time(&t);
    printf("C-Bot: Current system time is: %s", ctime(&t));
}
```

What this does:

- Gets system time.
 - Prints it as a readable string.
-

11. Telling a Random Fact

```
else if (strcmp(input, "tell me a fact") == 0) {
    const char *facts[] = {...};
    randomReply(facts, 3);
}
```

What this does:

Gives a random interesting fact.

12. Motivational Messages

```
else if (strcmp(input, "motivate me") == 0) {  
    const char *mot[] = {...};  
    randomReply(mot, 3);  
}
```

What this does:

Sends a random motivational message.

13. “love you bot”

```
else if (strcmp(input, "love you bot") == 0) {  
    printf("C-Bot: Love you too, %s ;) But I'm only 0s and 1s.\n",  
name);  
}
```

What this does:

Gives a personalized fun reply.

14. Help Menu

```
else if (strcmp(input, "help") == 0) {  
    printf("C-Bot Commands:\n");  
    printf("  hi / hello\n");  
    printf("  how are you\n");  
    printf("  time\n");  
    printf("  tell me a fact\n");  
    printf("  motivate me\n");  
    printf("  love you bot\n");  
    printf("  joke\n");  
    printf("  bye\n");  
}
```



```
}
```

What this does:

Shows all available commands.

15. Jokes

```
else if (strcmp(input, "joke") == 0) {  
    const char *jokes[] = {...};  
    randomReply(jokes, 3);  
}
```

What this does:

Prints a random joke.

16. Exit the Chat

```
else if (strcmp(input, "bye") == 0) {  
    printf("C-Bot: Goodbye %s! Take care :)\n", name);  
    break;  
}
```

What this does:

Ends the loop and exits the program.

17. Unknown Input Handler

```
else {  
    const char *unknown[] = {"I didn't understand that, try 'help'.",  
...};  
    randomReply(unknown, 3);  
}
```

```
}
```

What this does:

Shows random reply if input doesn't match any command.

18. Program End

```
return 0;
```

What this does:

Ends the program successfully.

Testing & Results:

```
C-Bot: Hello! I'm your smart Chatbot.
C-Bot: What's your name?
You: Vipransh
C-Bot: Nice to meet you, Vipransh ! Type 'help' to see options.

You: help
C-Bot Commands:
    hi / hello
    how are you
    time
    tell me a fact
    motivate me
    love you bot
    joke
    bye
You: hello
C-Bot: Hello! How's it going?
You: time
C-Bot: Current system time is: Sun Nov 30 22:28:45 2025
You: joke
C-Bot: Why do C programmers love Halloween? Too many bugs XD
You: love you bot
C-Bot: Love you too, Vipransh ;) But I'm only 0s and 1s.
You: motivate me
C-Bot: You're stronger than you think! :)
You: how are you
C-Bot: I'm good! Just chilling in the RAM :D
You: tell me a fact
C-Bot: Bananas are berries but strawberries aren't!
You: bye
C-Bot: Goodbye Vipransh ! Take care :)
PS C:\Users\vipra\OneDrive\Documents\C LANG\C Programming> █
```

Conclusion & Future Work:

Conclusion:

The project successfully demonstrates a basic console-based chatbot using C. The chatbot can recognize several predefined user commands, generate dynamic responses using randomization, display system time, and maintain a simple interactive conversation loop. It illustrates core programming concepts such as loops, conditional branching, string handling, use of standard libraries, and basic user interface design in a text console.

Future Work:

- Add case-insensitive input handling so that commands like "Hi" or "HELLO" are accepted.
- Expand the command set with more conversation topics and richer responses.
- Replace `system("cls")` with a more portable or library-based screen-handling method to support non-Windows platforms.
- Store responses and commands in external files or data structures for easier modification.
- Implement pattern matching or simple NLP techniques to handle more flexible user inputs instead of only fixed commands.

References:

- Class notes by Dr. Tanu Singh
- Let Us C by Yashvant Kanetkar
- GeeksforGeeks